

Question 1 (20 marks):

/20

1.a:

/2

Mark the following statements true or false:

- (a) ☐ False while loops will be endless without a break statement
- (b) ☐ True a variable value can change over the duration of a program
- (c) ☐ False the code `x = 3` is a comparison of x with the value of 3
- (d) ☐ False python does not compile code

1.b:

/2

Mark the following statements true or false:

- (a) ☐ True `g(f(x))`. The function `f()` is evaluated first
- (b) ☐ True tuples are immutable in python
- (c) ☐ False a dictionary is a collection that cannot contain duplicate values
- (d) ☐ False a stack will count the number of times a loop has executed

1.c:

What is the value of m after `m = int(9/2)` is executed?

/1

Put your answer here:

4

1.d:

What is the value of b after the following statements are executed?

/1

```
1 p = (25 + 5) > 30 and (50 - 20 == 30)
2 b = (not p) and (20 < 21)
```

Put your answer here:

True.

COMP9001; Write your SID:

1.e:

/2

Create a variable and initialise it to a list with 4 numbers, 10, 20, 30 and 40:

`a = [10, 20, 30, 40];` or `a = list(range(10, 50, 10))`

1.f:

/2

What is the output after the following statements are executed?

```
1 if (18 - 9) > 7:
2     print("Msg 3")
3 elif (18 - 9) > 2:
4     print("Msg 5")
5 print("Msg 2")
```

Put your answer here:

Msg 3 followed by Msg 2

1.g:

/2

When would you use logical operators in an if statement? Give an example

to control which code to execute next
based on value comparisons or a boolean
expression

1.h:

/2

Create a variable and initialise it as a tuple with 4 objects, 20, 'Jim', None and 30.0

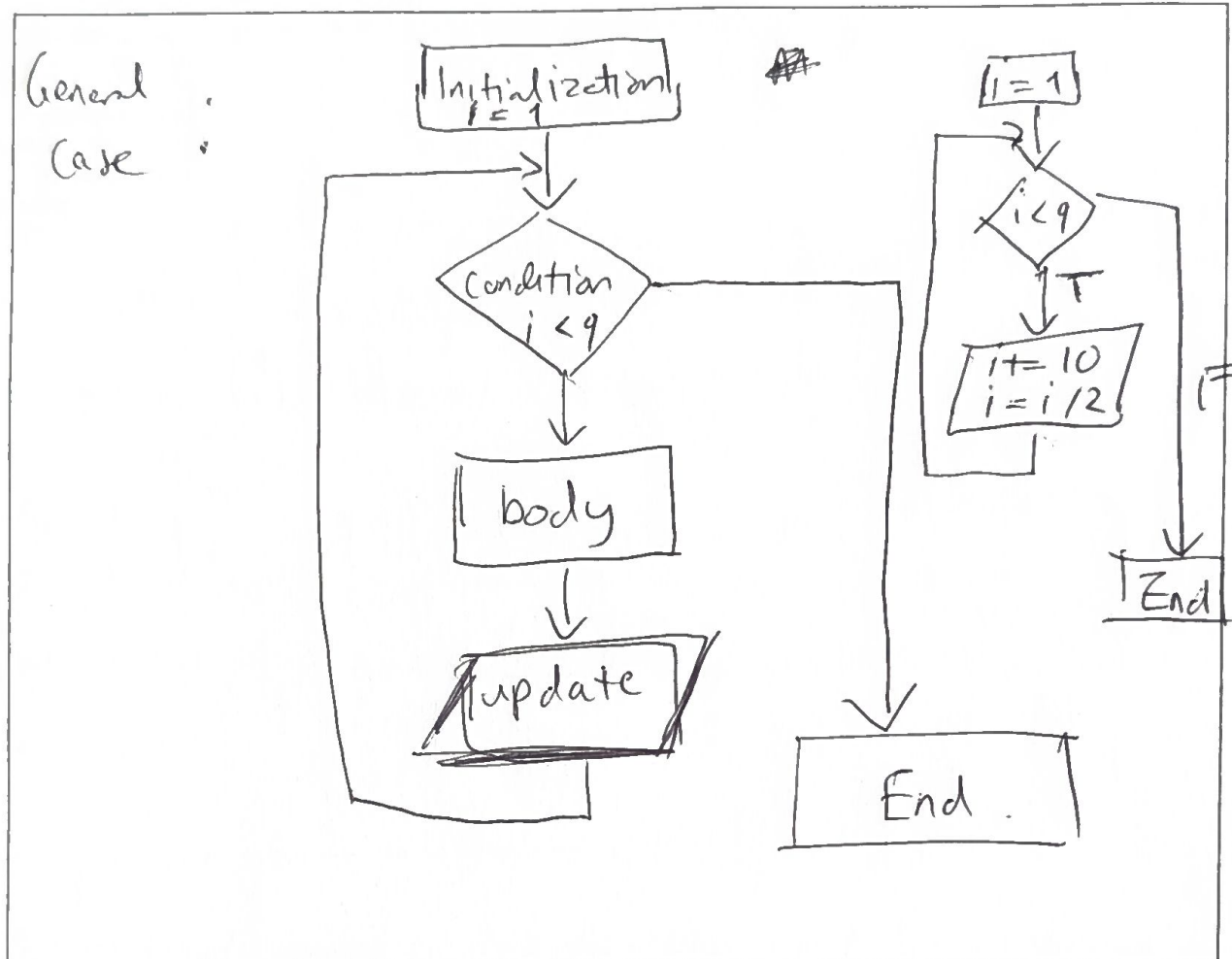
`a = (20, 'Jim', None, 30.0)`

1.i:

/4

Given the following loop, draw a flowchart / control flow diagram

```
1 i = 1
2 while i < 9:
3     i = i + 10
4     i = i / 2
5 print("Tony says " + str(i))
```



1.j:

/2

Draw the truth table for the and operator

A	B	A and B
T	T	T
T	F	F
F	T	F
F	F	F

COMP9001; Write your SID:

Question 2 (20 marks):

/20

2.a:

/6

We require a program to read all integers from the command line and print the minimum value.
The program will be run like this:

```
python print_min.py 34 1 25 -2 0 60 37 36 89 -43 18 80 49 19 -50
```

The following code is the entire contents of print_min.py. Find as many *different* errors (of any kind, syntax, or logical) as you can. Assume that k are intended to be the list of command line arguments. For each error state what line(s) it is on and why it is an error. *Putting just the line number with no explanation will get no mark.*

```
1 def output_min(k)
2     a = len(k)
3
4     for i in range(10)
5         i += 1
6         if k[i] > min_v:
7             min_v = k[i]
8         else
9             min_v = a
```

There are at least 8 errors, you need to identify 6 for full marks.

- (a) line: ; error is:
- (b) line: ; error is:
- (c) line: ; error is:
- (d) line: ; error is:
- (e) line: ; error is:
- (f) line: ; error is:
- (g) line: ; error is:
- (h) line: ; error is:

2.b:

/2

What could possibly go wrong with the following code when it is executed? Describe two of the errors and explain what kind of error each will cause. Use line numbers in your answer.

```
1 import sys
2
3 def so_wrong(contents):
4     x = int(contents[1])
5     new_contents = []*x
6     i = 0
7     while i < len(contents):
8         new_contents[i] = 1.0 / (5 - i)
9         i += 1
10
11 so_wrong(sys.argv)
```

i

If the number of arguments is less than 1 then contents[0] will throw an IndexError.

ii

If the argument is not a correctly formed int value, then the int() throw an exception (ValueError) too.

2.c:

/2

Mark the following statements true or false:

- (a) ☐ False The error keyword is used to create a new exception
- (b) ☐ True Syntax errors are automatically detected by the compiler
- (c) ☐ False Exceptions will always crash your program, even if they are caught
- (d) ☐ True Opening a file for reading will always be a successful

(d) should be 'False' as opening a file for reading will NOT always be succesful

Question 3 (15 marks):

/15

3.a:

/7

Write a module called Car with three functions and two global variables. Global variables colour is type str, and top_speed is type float. Their initial values are "none" and -1.337

Function initialise(p_colour, p_top_speed) sets the global variable. Each parameter should be type checked separately. If the type of a parameter is incorrect, then your code should verify and set each of these to their default value. If parameter p_colour is not type str, set colour → "white". If parameter p_top_speed is not type float, set top_speed → 120. Function is_fast() returns True if global variable top_speed is greater than or equal to 200, otherwise False.

Function is_super_fast() returns True if the global variable colour is red and also has global variable top_speed value greater than or equal to 200, otherwise False.

Hint: use isinstance for type checking

```
colour = "none"
top_speed = -1.337

def initialise(p_colour, p_top_speed):
    global colour, top_speed
    if isinstance(p_colour, str):
        colour = p_colour
    else:
        colour = 'white'

    if isinstance(p_top_speed, float):
        p_top_speed
        top_speed = p_top_speed
    else:
        top_speed = 120

def is_fast():
    return top_speed >= 200

def is_super_fast():
    return colour == 'red' and is_fast()
```

3.b:

Write a program called `divisibles` that takes n strings from command line arguments. A valid divisible includes a forward slash symbol `/` in between two numbers (numerator and denominator). For each valid divisible, your program should output the calculated floating point number of division. A string is not a divisible, and should be ignored when: 1) the string does not contain exactly one forward slash, or 2) the denominator or numerator could not be converted to a number, or 3) the division is unsuccessful.

Hint: use `sys` module, string method `split`, and exception handling

Examples

```
python divisibles 1/2 abc n/m 3.0/2 6/3 3/3/1
0.5 1.5 2
```

```
python divisibles 2/4 abc n/m 10/2 p/2 3/0 "a 4/2 b"
0.5 5.0
```

```
import sys
for e in sys.argv:
    spl = e.split('/')
    if len(spl) == 2:
        left = 0
        right = 0
        try:
            left = float(spl[0])
            right = float(spl[1])
            print(left/right)
        except:
            pass
```